# Comparative Lab: First Plots in R vs Python

Workshop 1 — Module 10

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

# Module Roadmap

1. Load the Google Play Store dataset in R and Python
2. Produce six chart types side by side: bar, histogram, scatter, boxplot, line, stacked bar
3. Apply Modules 1–9 principles to every chart
4. Build a six-panel EDA dashboard in both languages

**Learning Outcome**
You will produce identical, publication-quality charts in both R and Python, establishing the comparative fluency that continues through all nine workshops.

# Data Loading

# Loading the Dataset: R vs Python

## R (tidyverse)

```r
library(tidyverse)

apps <- read_csv(
  "googleplaystore.csv")

# Quick inspection
dim(apps)        # 10841 x 13
glimpse(apps)
summary(apps$Rating)

# Clean
apps_clean <- apps |>
  filter(Type %in%
    c("Free", "Paid")) |>
  mutate(
    Reviews = as.numeric(Reviews)
        )
```

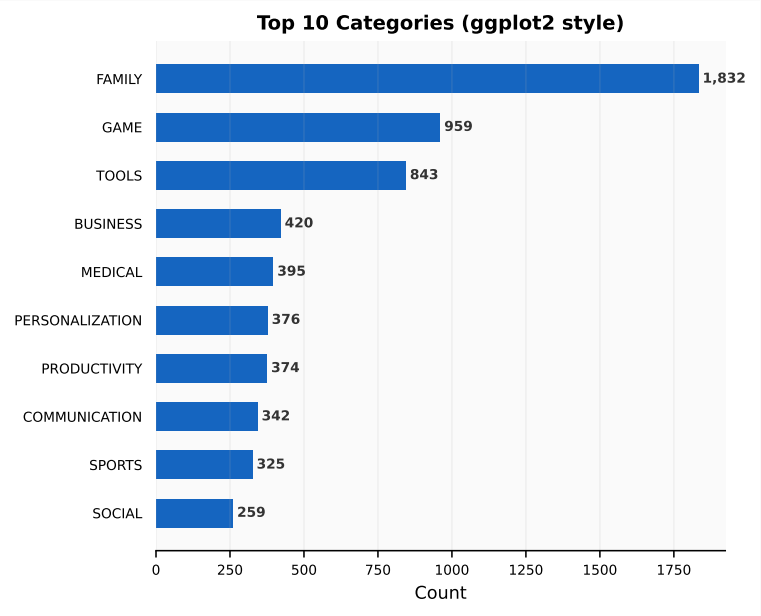
## Python (pandas)

```python
import pandas as pd
import numpy as np

apps = pd.read_csv(
    "googleplaystore.csv")

# Quick inspection
print(apps.shape) # (10841, 13)
print(apps.info())
print(apps["Rating"].describe())

# Clean
apps_clean = (apps
    .query("Type in ['Free','Paid
        ']")
    .assign(Reviews=lambda d:
        pd.to_numeric(
            d["Reviews"],
            errors="coerce")))
```

# Chart 1: Horizontal Bar

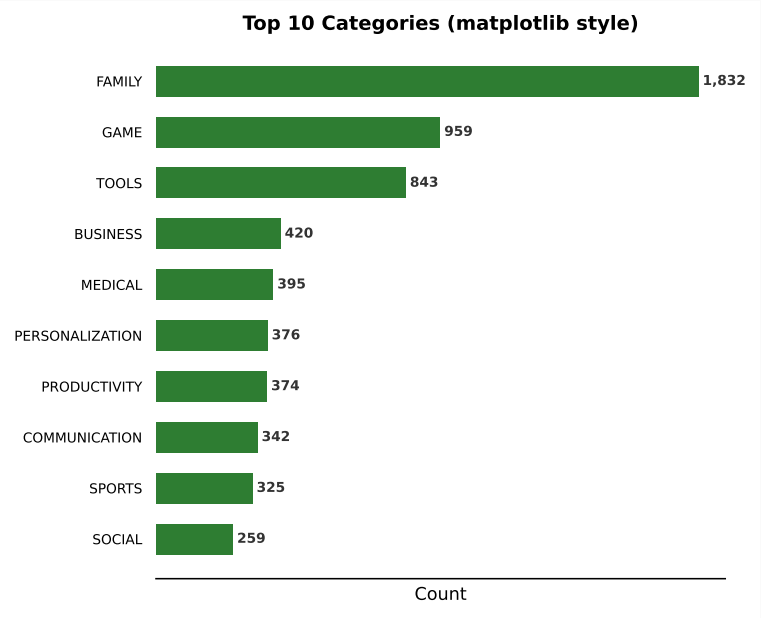
# Bar Chart — R (ggplot2)

```r
apps_clean |>
  count(Category, sort = TRUE) |>
  slice_max(n, n = 10) |>
  mutate(Category =
    fct_reorder(Category, n)) |>
  ggplot(aes(x = Category,
             y = n)) +
  geom_col(fill = "#1565C0",
           width = 0.6) +
  geom_text(aes(label =
    scales::comma(n)),
    hjust = -0.15, size = 3,
    fontface = "bold") +
  coord_flip() +
  theme_minimal() +
  theme(panel.grid.major.y =
    element_blank()) +
  labs(x = NULL, y = "Count",
    title = "Top 10 Categories")
```



# Bar Chart — Python (matplotlib)

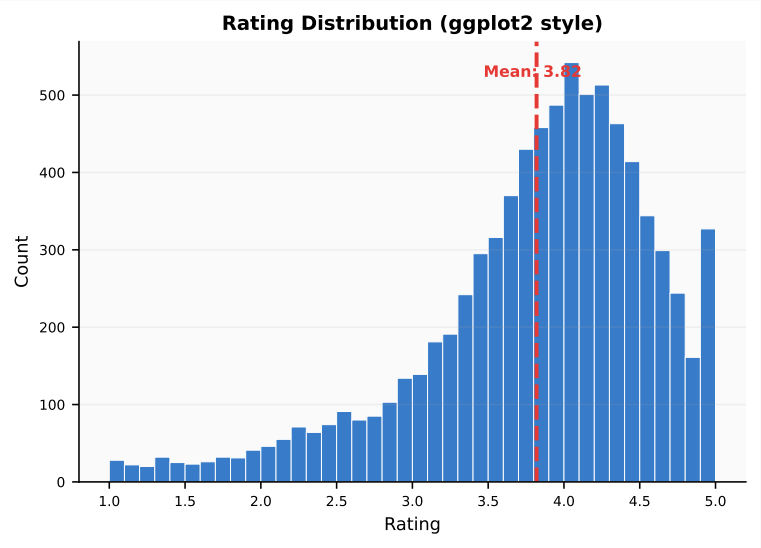
```python
top10 = (apps_clean
    .value_counts("Category")
    .head(10)
    .sort_values())

fig, ax = plt.subplots(
    figsize=(6, 5))
ax.barh(top10.index, top10.values,
    color="#2E7D32", height=0.6)

for i, v in enumerate(top10.values)
    :
    ax.text(v + 15, i, f"{v:,}",
        va="center", fontsize=7,
        fontweight="bold")

for sp in ["top","right","left"]:
    ax.spines[sp].set_visible(False
        )
ax.set_xticks([])
ax.tick_params(left=False)
ax.set_title("Top 10 Categories",
    fontweight="bold")
plt.tight_layout()
fig.savefig("bar.pdf")
```



# Chart 2: Histogram

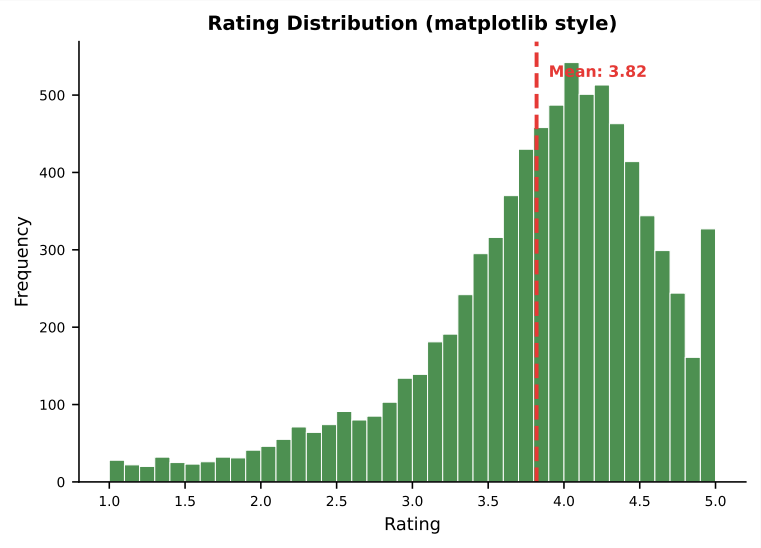
# Histogram — R (ggplot2)

```r
mean_r <- mean(apps_clean$Rating,
               na.rm = TRUE)

ggplot(apps_clean,
       aes(x = Rating)) +
  geom_histogram(
    bins = 40,
    fill = "#1565C0",
    color = "white",
    linewidth = 0.3) +
  geom_vline(
    xintercept = mean_r,
    color = "#E53935",
    linewidth = 1,
    linetype = "dashed") +
  annotate("text",
    x = mean_r - 0.3,
    y = Inf, vjust = 1.5,
    label = paste0("Mean: ",
      round(mean_r, 2)),
    fontface = "bold",
    color = "#E53935", size = 3) +
  theme_minimal() +
  labs(title = "Rating Distribution
       ",
       x = "Count")
```



# Histogram — Python (matplotlib)

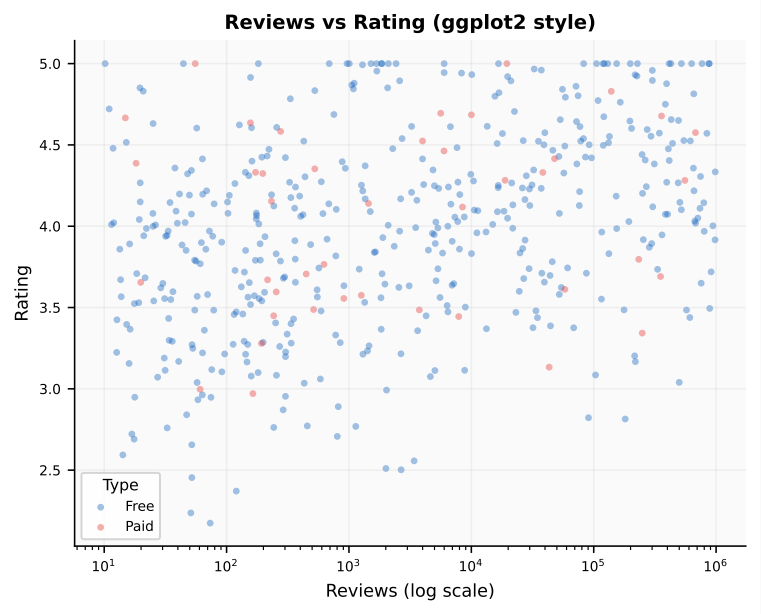
```
ratings = apps_clean["Rating"].
    dropna()
mean_r = ratings.mean()

fig, ax = plt.subplots(figsize
    =(6,4))
ax.hist(ratings, bins=40,
    color="#2E7D32",
    edgecolor="white", lw=0.3)

ax.axvline(x=mean_r,
    color="#E53935", lw=2,
    linestyle="--")
ax.text(mean_r + 0.05,
    ax.get_ylim()[1] * 0.92,
    f"Mean: {mean_r:.2f}",
    fontsize=8, color="#E53935",
    fontweight="bold")

ax.set_xlabel("Rating")
ax.set_ylabel("Frequency")
ax.set_title("Rating Distribution",
    fontweight="bold")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(
    False)
```



# Chart 3: Scatterplot

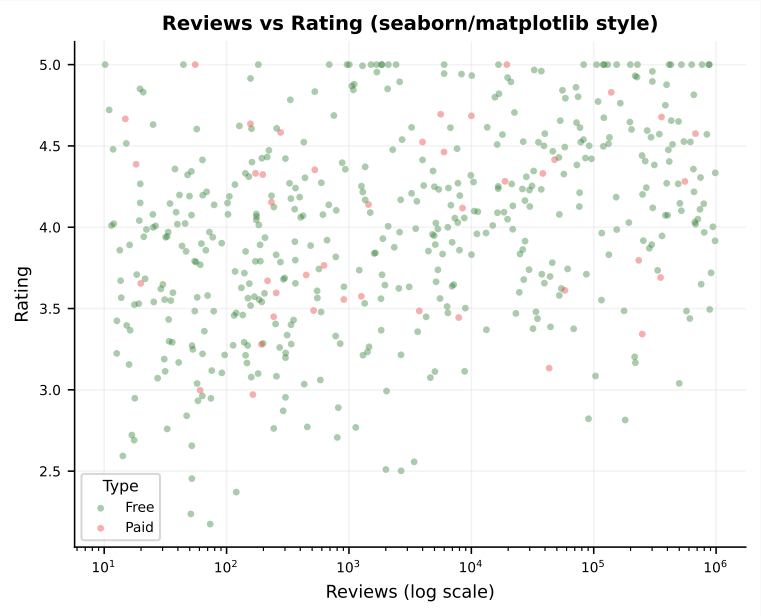
# Scatter — R (ggplot2)

```r
apps_clean |>
  filter(!is.na(Rating),
         Reviews > 0) |>
  ggplot(aes(x = Reviews,
             y = Rating,
             color = Type)) +
  geom_point(alpha = 0.3,
             size = 0.8) +
  scale_x_log10(
    labels = scales::comma) +
  scale_color_manual(
    values = c(Free = "#1565C0",
               Paid = "#E53935")) +
  theme_minimal() +
  labs(
    title = "Reviews vs Rating",
    x = "Reviews (log scale)",
    y = "Rating")
```



# Scatter — Python (matplotlib)

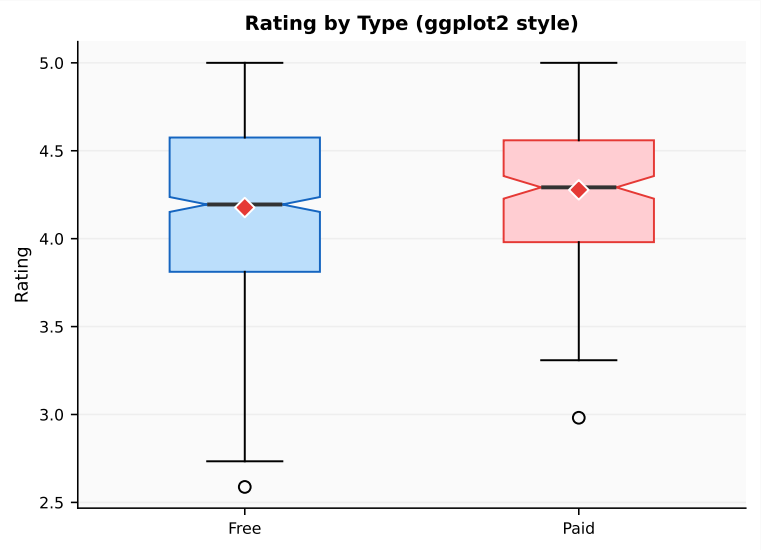
```python
df = apps_clean.dropna(
    subset=["Rating"])
df = df[df["Reviews"] > 0]

fig, ax = plt.subplots(figsize
    =(6,5))
palette = {"Free":"#2E7D32",
           "Paid":"#E53935"}

for t, c in palette.items():
    mask = df["Type"] == t
    ax.scatter(
        df.loc[mask, "Reviews"],
        df.loc[mask, "Rating"],
        s=8, c=c, alpha=0.3,
        edgecolors="none",
        label=t)

ax.set_xscale("log")
ax.set_xlabel("Reviews (log)")
ax.set_ylabel("Rating")
ax.set_title("Reviews vs Rating",
    fontweight="bold")
ax.legend(fontsize=7)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(
```



# Chart 4: Boxplot

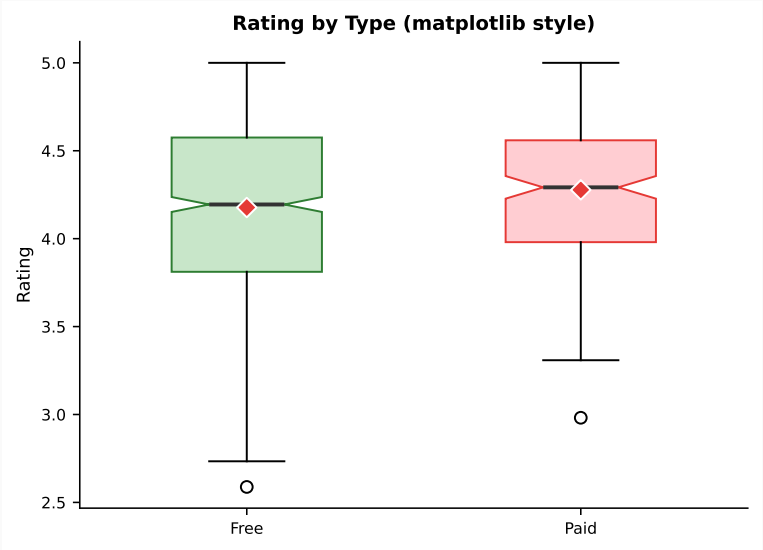
# Boxplot — R (ggplot2)

```r
apps_clean |>
  filter(!is.na(Rating)) |>
  ggplot(aes(x = Type,
             y = Rating,
             fill = Type)) +
  geom_boxplot(
    width = 0.5,
    notch = TRUE,
    outlier.alpha = 0.2) +
  stat_summary(
    fun = mean,
    geom = "point",
    shape = 18, size = 3,
    color = "#E53935") +
  scale_fill_manual(
    values = c(
      Free = "#BBDEFB",
      Paid = "#FFCDD2")) +
  theme_minimal() +
  theme(legend.position =
    "none") +
  labs(title = "Rating by Type",
       y = "Rating", x = NULL)
```



# Boxplot — Python (matplotlib)

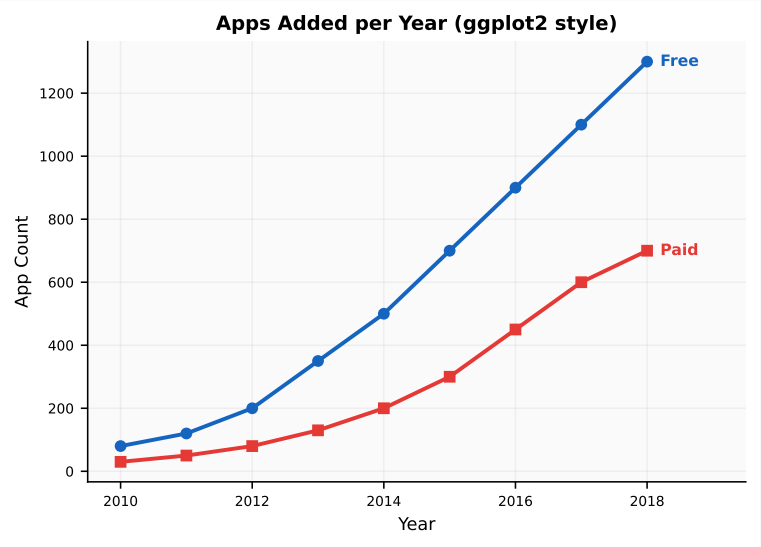
```
free = df.loc[
    df["Type"]=="Free","Rating"]
paid = df.loc[
    df["Type"]=="Paid","Rating"]

fig, ax = plt.subplots(figsize
    =(5,4))
bp = ax.boxplot(
    [free, paid],
    labels=["Free", "Paid"],
    patch_artist=True,
    widths=0.45, notch=True)

bp["boxes"][0].set_facecolor(
    "#C8E6C9")
bp["boxes"][1].set_facecolor(
    "#FFCDD2")

# Overlay mean points
means = [free.mean(), paid.mean()]
ax.scatter([1, 2], means,
    c="#E53935", s=50, zorder=5,
    marker="D", edgecolors="white")

ax.set_ylabel("Rating")
ax.set_title("Rating by Type
```



# Chart 5: Line Chart

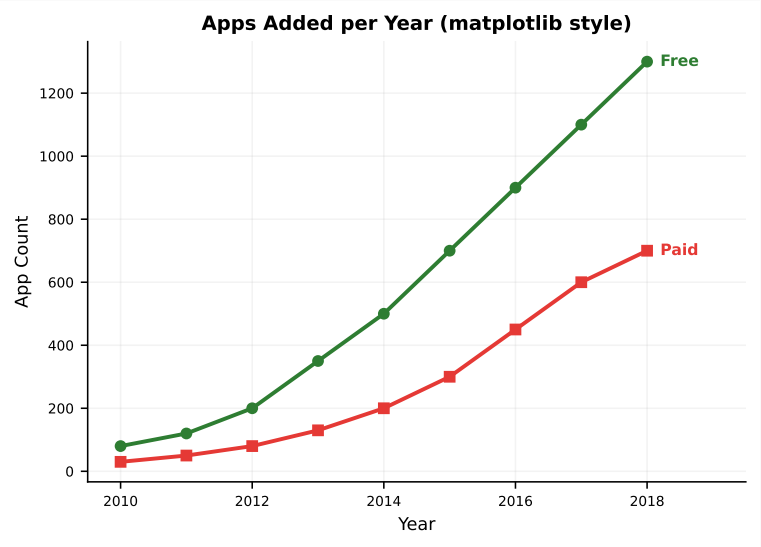
# Line Chart — R (ggplot2)

```r
apps_clean |>
  mutate(year = year(
    mdy(`Last Updated`))) |>
  count(year, Type) |>
  filter(year >= 2010) |>
  ggplot(aes(x = year, y = n,
             color = Type)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 2) +
  # Direct labels at endpoint
  geom_text(
    data = . %>%
      filter(year == max(year)),
    aes(label = Type),
    nudge_x = 0.3, size = 3,
    fontface = "bold") +
  scale_color_manual(
    values = c(Free = "#1565C0",
               Paid = "#E53935")) +
  theme_minimal() +
  theme(legend.position = "none") +
  labs(title = "Apps Added/Year",
       x = "Year", y = "Count")
```



# Line Chart — Python (matplotlib)

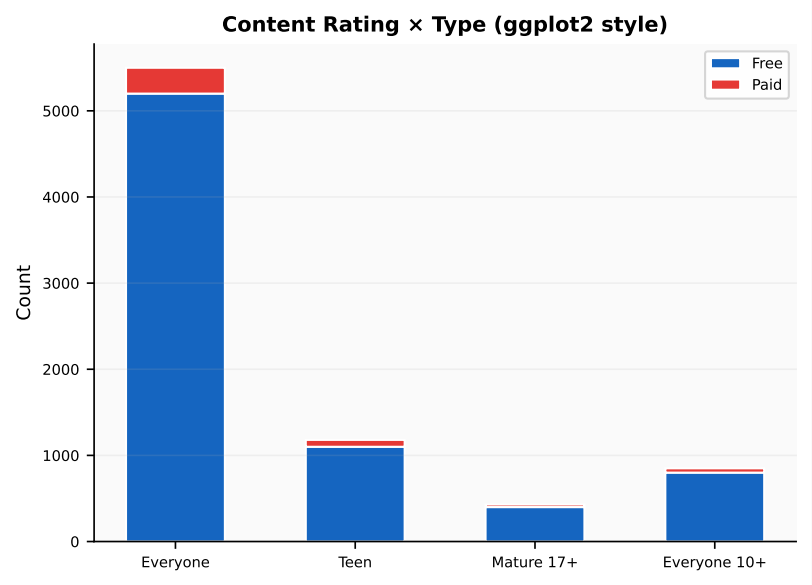
```
apps_clean["year"] = pd.to_datetime
    (
    apps_clean["Last Updated"],
    format="%B %d, %Y",
    errors="coerce").dt.year

yearly = (apps_clean
    .groupby(["year","Type"])
    .size()
    .reset_index(name="n")
    .query("year >= 2010"))

fig, ax = plt.subplots(figsize
    =(6,4))
for t, c in [("Free","#2E7D32"),
             ("Paid","#E53935")]:
    sub = yearly.query("Type==@t")
    ax.plot(sub["year"], sub["n"],
        "o-", color=c, lw=2, ms=5)
    ax.text(sub["year"].iloc
        [-1]+0.2,
        sub["n"].iloc[-1], t,
        fontsize=8, fontweight="
            bold",
        color=c, va="center")
```



# Chart 6: Stacked Bar

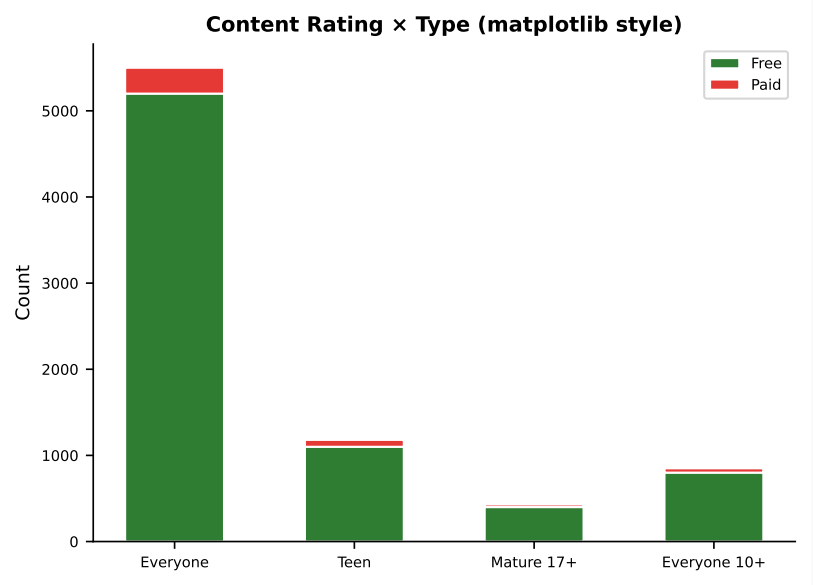
# Stacked Bar — R vs Python

## R (ggplot2)




```r
ggplot(apps_clean,
  aes(x = `Content Rating`,
      fill = Type)) +
  geom_bar(position = "stack") +
  scale_fill_manual(values =
    c(Free="#1565C0",
      Paid="#E53935")) +
  theme_minimal()
```

## Python (matplotlib)




```python
ct = pd.crosstab(
  apps_clean["Content Rating"],
  apps_clean["Type"])
ct.plot.bar(stacked=True,
  color=["#2E7D32","#E53935"],
  edgecolor="white", ax=ax)
```

# Syntax Cheat Sheet

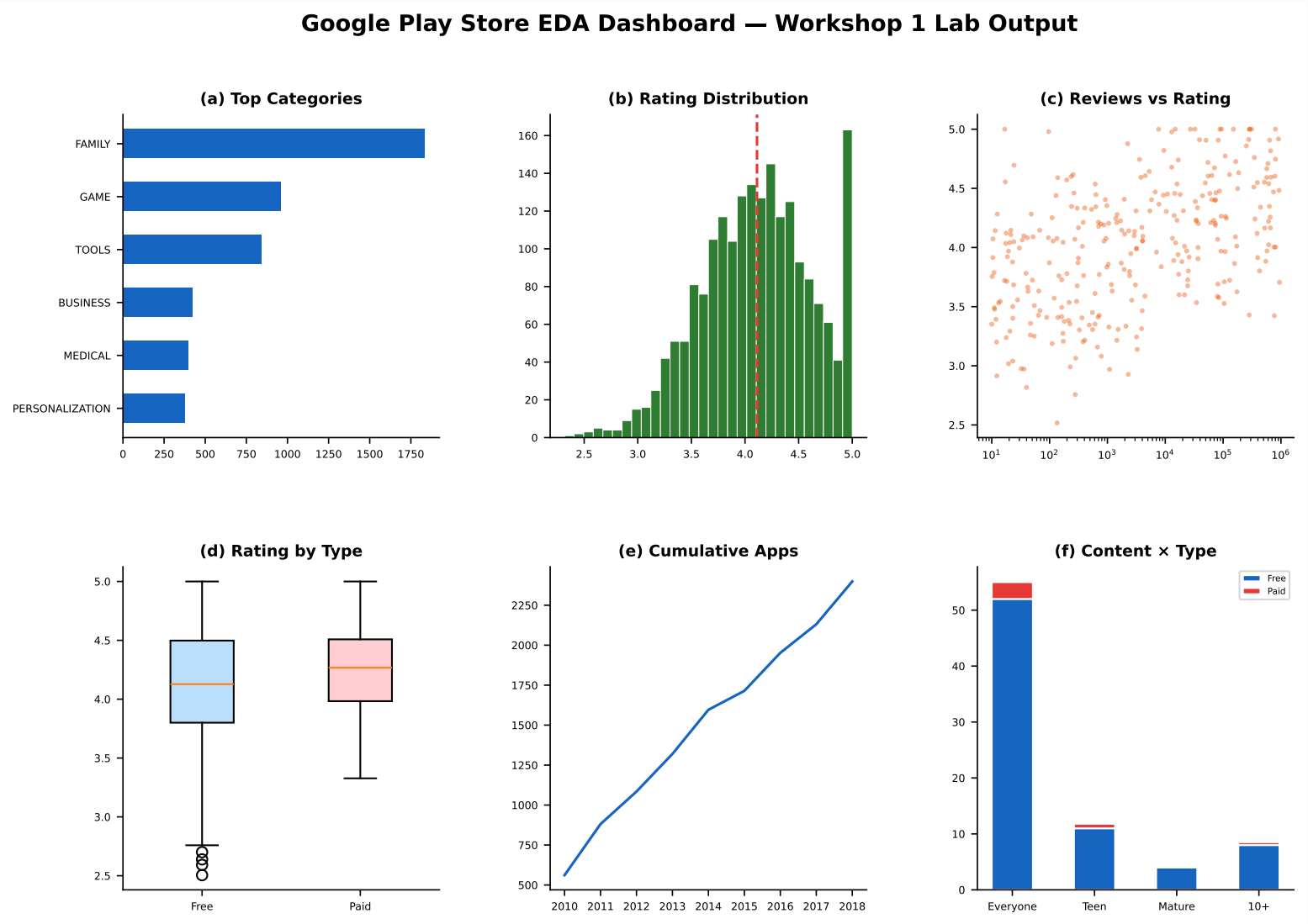
# R ↔ Python Complete Comparison

**R ↔ Python Syntax Comparison: Complete Cheat Sheet**

| Operation | R (tidyverse + ggplot2) | Python (pandas + matplotlib) |
|---|---|---|
| Load data | `read_csv("data.csv")` | `pd.read_csv("data.csv")` |
| Filter rows | `filter(df, Type == "Free")` | `df.query("Type == 'Free'"` |
| Count | `count(Category)` | `value_counts("Category")` |
| Sort | `arrange(desc(n))` | `.sort_values(ascending=False)` |
| Bar chart | `geom_col(fill='blue')` | `ax.barh(cats, vals, color='blue')` |
| Histogram | `geom_histogram(bins=30)` | `ax.hist(x, bins=30)` |
| Scatter | `geom_point(aes(color=Type))` | `ax.scatter(x, y, c=colors)` |
| Boxplot | `geom_boxplot(fill=Type)` | `ax.boxplot([g1,g2], patch_artist)` |
| Line | `geom_line(aes(color=grp))` | `ax.plot(x, y, color='blue')` |
| Save | `ggsave("plot.pdf", w=7, h=5)` | `fig.savefig("plot.pdf")` |

# Lab Dashboard

# Final Output: Six-Panel EDA Dashboard




**Data Insight**

This dashboard combines all six chart types from the lab. In practice you

# Synthesis

# Workshop 1: What You Now Know

| Module | Topic | Core Skill |
|---|---|---|
| M01 | Why Visualize | Anscombe, Datasaurus |
| M02 | Tufte's Principles | Data-ink, lie factor |
| M03 | Visual Perception | Pre-attentive, Gestalt |
| M04 | Color & Accessibility | Viridis, CVD, grey-accent |
| M05 | Typography & Layout | Hierarchy, direct labels |
| M06 | Design Process | Cairo, Munzner, pipeline |
| M07 | Critique & Redesign | 4-quadrant, SBI feedback |
| M08 | R Environment | Base R graphics |
| M09 | Python Environment | matplotlib OO API |
| M10 | Comparative Lab | R/Python parity |

# Key Takeaways

1. **R and Python produce identical results** — the choice is context-dependent, not quality-dependent.
2. The **six chart types** (bar, histogram, scatter, boxplot, line, stacked bar) cover the vast majority of EDA needs.
3. Every chart applied principles from M01–M07: sorted bars (M02), log scales (M03), colour by type (M04), direct labels (M05), Tufte cleanup (M02), audience-appropriate titles (M06).
4. **Workshop 2** builds on this foundation with the Grammar of Graphics: ggplot2 layers (R) and plotnine/seaborn (Python).

# Workshop 1 Homework Summary

**Due: Before Workshop 2**
Combine all module homeworks into a single PDF portfolio:

- 10 module assignments (Parts A–C each)
- All R scripts (`.R`) and Python scripts (`.py`)
- All exported figures (PDF vector + PNG 300 dpi)

Total weight: 5% of course grade.

**Next**: Workshop 2 — The Grammar of Graphics (M01: Wilkinson's Theory)